

Software requirements

The CorDapp development software requirements are:

- Java 8 JVM (at least version 8u171)
- Gradle
- IntelliJ IDEA

Detailed setup instructions are available at <https://docs.corda.net/getting-set-up.html>.

The CorDapp can be executed in two ways. One is to execute it via the terminal and the other is to run it via IntelliJ.

Building your own CorDapp

CorDapp generally defines three items, as listed below:

- **States:** These are shared facts that are written to the ledger.
- **Flows:** This refers to the procedure to be carried out for specific updates to the ledger.
- **Contracts:** These are constraints on how the state of a given type can evolve over time.

The CorDapp template can be downloaded with the following code.

For Java:

```
git clone https://github.com/corda/cordapp-template-java.git;
cd cordapp-template-java
```

For Kotlin:

```
git clone https://github.com/corda/cordapp-template-kotlin.git;
cd cordapp-template-kotlin
```

Let's take a look at a sample of the State and Flow code. A sample *TemplateState.java* is shown below:

```
// Add this import:
import net.corda.core.identity.Party;

// Replace TemplateState's definition with:
public class IOUState implements ContractState {
    private final int value;
    private final Party lender;
    private final Party borrower;

    public IOUState(int value, Party lender, Party borrower)
    {
        this.value = value;
        this.lender = lender;
        this.borrower = borrower;
    }

    public int getValue() {
        return value;
    }

    public Party getLender() {
```

```
        return lender;
    }

    public Party getBorrower() {
        return borrower;
    }

    @Override
    public List<AbstractParty> getParticipants() {
        return Arrays.asList(lender, borrower);
    }
}
```

A sample of *FlowLogic* is shown below:

```
// Add these imports:
import net.corda.core.contracts.Command;
import net.corda.core.identity.Party;
import net.corda.core.transactions.SignedTransaction;
import net.corda.core.transactions.TransactionBuilder;

// Replace Initiator's definition with:
@InitiatingFlow
@StartableByRPC
public class IOUFlow extends FlowLogic<Void> {
    private final Integer iouValue;
    private final Party otherParty;

    /**
     * The progress tracker provides checkpoints indicating
     * the progress of the flow to observers.
     */
    private final ProgressTracker progressTracker = new
    ProgressTracker();

    public IOUFlow(Integer iouValue, Party otherParty) {
        this.iouValue = iouValue;
        this.otherParty = otherParty;
    }

    @Override
    public ProgressTracker getProgressTracker() {
        return progressTracker;
    }

    /**
     * The flow logic is encapsulated within the call()
     * method.
     */
    @Suspendable
    @Override
    public Void call() throws FlowException {
        // We retrieve the notary identity from the network
        map.
```